HARD Problem

Title

Amicable Chain Length up to N

Topics

Number Theory, Divisor Sum, Mathematical Cycles, Nested Loops, Simulation, Conditional Logic

Difficulty Level

Hard

Problem Detailed Statement (Scenario Based)

In an elite number theory research workshop, students explore special relationships between numbers known as **Amicable Numbers**.

Two numbers A and B are called **Amicable Numbers** if:

- Sum of proper divisors of A = B
- Sum of proper divisors of B = A
- A ≠ B

Example:

220 → divisors sum = 284
284 → divisors sum = 220

So (220, 284) is an amicable pair.

But the trainer now gives a more complex challenge:

Instead of just checking pairs, students must analyze **Amicable Chains**.

Definition:

1. Start from a number N.
2. Replace it with the sum of its proper divisors.
3. Repeat the process.

4. If the sequence eventually loops back to the original number, it forms an **Amicable Chain**.
5. The length of the loop is the chain length.

Example (amicable pair case):

220 → 284
284 → 220

Chain length = 2

Now, students are given a positive integer **N**.

They must:

1. Generate the divisor-sum sequence starting from N.
2. Detect whether a cycle occurs.
3. If the cycle returns to N → it is an amicable chain.
4. Print the length of the cycle.
5. If no cycle returns to N → print -1.

Students must:

- Compute proper divisors using loop logic.
- Avoid storing entire sequences using large data structures (bonus challenge).
- Implement cycle detection manually.
- Handle large intermediate values carefully.
- Stop if number exceeds a reasonable bound (to avoid infinite growth).

Edge Cases:

- Prime numbers (sum becomes 1)
- Perfect numbers (loop length = 1)
- Amicable pair (length = 2)
- Longer chains (rare but possible)

Input Format

`N`

Constraints:

`2 ≤ N ≤ 100000`

Output Format

- Print the length of the amicable chain.
- If no valid amicable chain exists, print:

```
-1
```

Examples

Example 1

Input:

```
220
```

Output:

```
2
```

Example 2

Input:

```
6
```

Explanation:
6 → divisors sum = 6

Perfect number → cycle length 1

Output:

```
1
```

Example 3

Input:

```
10
```

10 → 8 → 7 → 1 → 0

No cycle returning to 10

Output:

```
-1
```